

HOLLOWDEEP

Game Design Document — Capstone

1. Executive Summary

The Game

HOLLOWDEEP is a party-based, turn-based tactical dungeon crawler for PC, built in Unity. A fixed four-hero party — two Fighters and two Priests — descends into a buried barrow-complex to recover the Sunstone, a relic that once held back the dark beneath the world. The game blends HeroQuest's room-by-room board flow with Shadowdark's grim lethality and light-as-a-resource tension: every torch burns down round by round on a hard 30-round timer, and when it gutters out, the dark stops waiting.

The Core Loop

Enter a room → reveal it and its threats → resolve tactical grid combat or a hazard → loot fallen enemies and containers → decide whether to press deeper on a burning torch or retreat to the surface to bank progress before it gutters → repeat with rising danger, until the party reaches the Sunstone chamber and extracts, or is wiped out.

Win & Loss Conditions

Win: The party recovers the Sunstone from the deepest chamber of the delve AND at least one hero escapes back to the surface entrance with it.

Loss: The entire party is reduced to 0 HP with no one able to stabilize a downed ally (total party wipe), OR the party's torch timer reaches 0 with no fresh torch or Light spell available while still below ground, and a Dark Event resolves against them.

Three Game Design Pillars

- Thematic genre — Grim low-fantasy dungeon delving: torchlit stone, scavenged gear, and a world that does not owe the party a fair fight.
- Mechanic genre — Tactical grid combat framed by a room-by-room board flow, gated by a depleting Light resource that forces pace decisions.
- Unique feeling — Dread that compounds turn over turn: the player should feel each Light point spent, not just each hit point lost.

Comparisons

HeroQuest (room-by-room reveal and board pacing), Shadowdark RPG (torch-as-timer tension and lethality), Darkest Dungeon (resource attrition and dread), and my prototype build, a Unity turn-based tactics project since upgraded to Unity 6.4 (grid movement, action points, and turn order — the systems foundation this project extends).

Stretch Goals

- Additional playable classes beyond the two MVP classes
 - A town hub between delves (shop, rest, hire a replacement hero)
 - A second, deeper dungeon wing with a distinct hazard theme
 - Lightweight procedural room ordering for replayability
-

2. Game Mechanics

2.1 Player Actions

Verb	Input	What the player sees	The decision it enables
Move	Click a highlighted tile	Hero token steps across the grid, spending movement from its Action Points	Positioning for cover, flanking, or retreat before committing further actions
Attack	Click an enemy in range	A short resolve animation, damage numbers, and a hit/miss result	Trading Action Points now for damage, weighed against saving AP to reposition
Use Item	Open pack, click item	Item is consumed or equipped; effect applies immediately (heal, buff, oil for torches)	Spending a limited consumable now vs. hoarding it for a worse moment
Cast Ability	Select ability, click target/tile	Class-specific effect (e.g., Priest's Light spell relights the party's torch)	Spending a cooldown resource for a swing in the fight's momentum, or to stretch the torch timer
Loot	Click a defeated enemy or container	Item(s) drop into the party's shared inventory with a short pickup flourish	Stopping to loot everything (costs the action, and a tick of the torch timer) vs. pushing on and leaving loot behind
Manage Light	Click torch icon	The party lights a fresh torch; its 30-round timer resets and the 8-square light radius snaps back to full	Choosing to relight now (costs a torch, safer) or push on as the timer runs low (risk)
Search / Interact	Click a door, chest, or lever	Door creaks open and the next room is revealed; chest opens or triggers a trap check	Deciding whether to search fully (time and a timer tick) or move past quickly
End Turn / Rest	Click End Turn button	Turn passes to the next hero, then to enemies; a short Rest recovers a little HP between rooms	Banking remaining AP vs. holding position for a reaction next round

2.2 Moment-to-Moment Play

The party's torch reads 11 rounds left as they step into a collapsed shrine, two rooms deep. The door creaks open and three skeletal wardens are revealed clustered around a broken altar — all just inside the torch's 8-square radius, with a fourth shape still lurking in the dark beyond it. It's Bram the Fighter's turn: he spends two Action Points to move onto the chokepoint tile in the doorway and his last two on an attack, dropping a skeleton before it can act. The torch ticks down to 10. Then Sela the Priest acts — she could finish a wounded skeleton with Smite, or spend her action on a Light spell to reset the timer, since 10 rounds might not cover the rest of this room plus the walk back. She judges the fight nearly won and finishes the skeleton instead. Two rounds later the last enemy falls, the party spends an action to Loot the altar and the fallen skeletons, and with the torch now at 7 rounds, the player has to decide: press toward the Sunstone chamber on a dwindling timer, or fall back to relight before advancing further.

2.3 Systems as the Player Meets Them

2.3.1 Light

What the player experiences: A torch icon with a 30-segment countdown ring, ticking down by exactly one every round no matter what the party does that round. The lit torch also casts an 8-square radius on the grid — tiles inside it are visible and targetable, tiles outside sit in shadow until the party gets close.

The rules as the player learns them: Every round, in exploration or combat alike, the timer drops by 1. Lighting a fresh torch or a Priest casting the Light spell resets it to 30 and restores the full 8-square radius. At 0, the party is plunged into darkness and a Dark Event check triggers each round until someone relights.

Key values: 30 rounds per torch or Light spell; 8-square light radius; the party starts each delve with 2 torches plus the Priest's Light spell as a third light source; any additional torches or light sources are found and picked up in the dungeon, not bought beforehand.

2.3.2 Grid Combat & Action Points

What the player experiences: Combat drops onto a visible grid overlaid on the room. Each hero gets a small pool of Action Points per turn to spend on moving, attacking, or abilities, in any order.

The rules as the player learns them: Movement costs 1 AP per tile, attacks cost 2 AP, abilities cost AP plus a cooldown. Enemies act on their own turn using the same grid and line-of-sight rules.

Key values: Heroes have 4 AP per turn; most rooms host 2-4 enemies; a hero's melee range is 1 tile, most ranged attacks reach about half the room.

2.3.3 Party & Downed Recovery

What the player experiences: A fixed four-hero party — two Fighters and two Priests — shown as a portrait row with HP bars. A hero reduced to 0 HP falls but isn't dead — they're down, and an adjacent ally can spend a turn to stabilize them.

The rules as the player learns them: A downed hero left unstabilized for too many rounds, or hit again while down, dies permanently for that delve. The whole party wiping ends the run. Priests are the party's main stabilizers and Light-spell source; Fighters are the durable frontline that holds chokepoints while the torch timer runs.

Key values: Fighters run roughly 16-18 HP, Priests roughly 12-14 HP; stabilizing costs a full action and restores a small buffer of HP, not a full heal.

2.4 Progression, Failure & Recovery

Danger escalates through the delve on two tracks: room depth (later rooms hold tougher, more numerous enemies) and the torch timer (as it counts down toward 0, the odds and severity of Dark Events rise, and once it hits 0 those checks trigger every round). Failure at the encounter level is a hero going down, recoverable through stabilization; failure at the run level is a total party wipe or a Dark Event overwhelming the party with the torch spent, either of which ends the delve short of the Sunstone. The player's core recovery lever is the retreat decision from Section 2.2: banking progress by returning to the surface converts a risky push into a smaller, guaranteed gain, directly trading off against the win condition of reaching and extracting the Sunstone.

3. AI Architecture

Development work is organized into two crews — small groups of agents with a shared deliverable and a lead agent who resolves conflicts within the crew before anything is handed off — plus one standalone agent that reports directly to me. A crew exists here because the Systems and Content workstreams each need more than one agent working on shared state (a shared codebase for Systems, a shared level list for Content) at the same time; the lead's job is specifically to keep those parallel edits from colliding before I ever see them.

3.1 Agent Roster

Systems Crew

Systems Lead — Role: Breaks a feature request (e.g., “add the torch timer”) into scoped coding tasks for its two agents, and merges their output into the single Unity 6.4 project so their edits don't conflict.

Grid/Turn Agent — Role: Implements and tests the grid movement, turn order, and Action Point systems on top of my upgraded prototype build.

Light Timer Agent — Role: Implements the 30-round torch timer, the 8-square light radius, and the Dark Event trigger that fires when the timer hits 0.

Implementation detail: The timer decrements on the turn manager's own `OnRoundEnd` C# event, not a Coroutine — Coroutines run against Unity's frame-based Update loop, so a round that spans multiple frames (an animation, a delayed enemy action) risks the timer ticking more or less than once per round. Subscribing directly to `OnRoundEnd` keeps the countdown locked to game rounds rather than wall-clock time. At 0, the agent raises its own `OnDarkEventTriggered` event, which the combat and UI systems subscribe to independently — so nothing needs to poll the timer's value every frame to know when the dark closes in.

What the crew produces: C# scripts, prefab wiring, and small unit tests for turn/AP/timer logic, merged by the Systems Lead into a single working build.

Gameplay effect: The player gets responsive turn order, movement, and a torch timer that ticks and resets exactly on the rules in Section 2.3.1, without desyncs or soft-locks mid-combat.

What it does NOT do: Design encounters, tune balance numbers, or produce art.

Content Crew

Content Lead — Role: Reviews room, loot, and art submissions from its two agents and sequences them into the level list, rejecting any room that references a system the Systems Crew hasn't shipped yet.

Encounter Agent — Role: Drafts dungeon room layouts, enemy compositions, and loot tables as structured data (ScriptableObject/JSON).

Art/Asset Agent — Role: Sources and integrates free placeholder asset packs (sprites, tilesets, UI icons) matching the Shadowdark-grim visual target, rather than generating original art — free packs are just as usable for an MVP build and don't spend agent tokens on iterative image generation.

What the crew produces: Validated room/encounter definitions plus matching art, consumed by the Systems Crew's grid loader.

Gameplay effect: The player moves through rooms that feel hand-placed and visually coherent — varied enemy mixes and a difficulty curve toward the Sunstone chamber.

What it does NOT do: Write or modify gameplay code.

Standalone: QA/Balance Agent

Role: Reviews playtest logs from both crews' output and flags balance outliers in torch-timer pacing, combat difficulty, and Action Point economy.

What it produces: Short balance reports with suggested numeric deltas (e.g., “Room 4 burns the torch timer 20% faster than the target pace”).

Gameplay effect: The player experiences a difficulty curve that escalates on purpose rather than by accident.

What it does NOT do: Apply tuning changes itself, or report to either crew lead — every numeric change is approved by me and routed back to the relevant lead.

3.2 How the Crews Work Together

Within the Systems Crew, the Grid/Turn Agent and Light Timer Agent work against the same codebase in parallel; the Systems Lead is the one who merges their branches and resolves any overlap before the result becomes the shared baseline. That validated baseline is what the Content Crew builds against: the Content Lead only accepts an Encounter Agent room into the level list once it references enemy and tile types the Systems Lead has confirmed exist, and pairs each accepted room with matching sprites from the Art/Asset Agent by tile-type key. If an Encounter Agent room references something the Systems Crew hasn't shipped yet, the Content Lead bounces it back rather than passing it through broken. After each milestone build, the QA/Balance Agent reads playtest logs spanning both crews' output and sends its report to me directly; I route any approved tuning change back to the relevant crew lead — never straight to an individual agent — so each lead stays the single source of truth for their crew's codebase or dataset.

3.3 The Human in the Loop

I personally verify feel and fun: whether a room's pacing is tense in the way the pillars intend, whether combat reads clearly on the grid, and whether the torch timer's countdown actually feels like pressure rather than an ignorable UI element — none of which a crew can judge by running the build. My sign-off gates three checkpoints: accepting the Systems Lead's merged baseline before the Content Crew builds on it, accepting the Content Lead's finished rooms before they enter the level list, and approving any QA/Balance Agent tuning suggestion before it's applied.

4. Technical Strategy

4.1 Stack & Scope

Engine / language / key tools: Unity 6.4, C#, targeting PC (Windows). Grid/turn/Action Point foundation hand-built for my prototype build, then personally upgraded and extended from its original target version to Unity 6.4.

Semester scope (what ships): One full dungeon wing (6-8 rooms) ending at the Sunstone chamber, a fixed four-hero starting party (2 Fighters, 2 Priests), 3-4 enemy types, the 30-round torch timer with its 8-square light radius, downed/stabilize recovery, and functional win/loss conditions (Sections 2.1-2.4).

Explicitly deferred: Additional classes beyond the starting two, the town hub, procedural room generation, and any second dungeon wing — all listed as stretch goals in Section 1, not required for a complete, winnable build.

4.2 Crew Roles in the Build Plan

Five-week schedule: Discovery (Week 0, this GDD plus a Unity 6.4 codebase audit) → Core Systems (Weeks 1-2, Systems Crew) → Content & Combat (Weeks 2-3, Content Crew with Systems Crew loader support) → Testing (Week 4, QA/Balance Agent) → Polish & Release (Week 5). Mapped onto crews and deliverables below:

Phase	Weeks	Active crew/agent	Deliverable	Verified by
Discovery	0	None (planning only)	Finalized GDD, Unity 6.4 codebase audit	Me
Core Systems	1-2	Systems Crew	Grid movement, turn order, AP economy, torch timer running in a test scene	Me (playtest of raw systems)
Content & Combat	2-3	Content Crew, Systems Crew (loader support)	Full dungeon wing playable start-to-finish with placeholder-to-final art	Me (full delve playtest)
Testing	4	QA/Balance Agent	Balance report and applied tuning pass	Me (approve each tuning change)
Polish & Release	5	Systems Crew, Content Crew's Art/Asset Agent	Final build, bug fixes, UI polish	Me (milestone sign-off, expo to PDF/build)

4.3 Token Budget

Model(s) and access: Claude Code (Sonnet-class model) via a standard subscription, used for implementation and review sessions across both crews.

Estimated tokens per session: ~50,000 tokens (input + output combined). A “session” is one focused working block for a single agent — e.g., the Light Timer Agent implementing the torch countdown, or the Encounter Agent generating one batch of room definitions — including the code/context read in and the changes written out.

Sessions per week: × 5 (roughly one focused agent session per weekday, across whichever crew is active that phase)

Weeks: × 5

Total estimate: $\approx 50,000 \times 5 \times 5 = 1,250,000$ tokens across the project.

Broken down by phase and agent, the 25 sessions across the 5-week schedule split as follows:

Phase	Agent	Sessions	Tokens
Core Systems	Grid/Turn Agent	5	250,000
Core Systems	Light Timer Agent	3	150,000
Core Systems	Systems Lead (merge/review)	2	100,000
Content & Combat	Encounter Agent	4	200,000
Content & Combat	Art/Asset Agent	2	100,000
Content & Combat	Content Lead (review/sequencing)	1	50,000
Content & Combat	Systems Crew (loader support)	1	50,000
Testing	QA/Balance Agent	4	200,000
Polish & Release	Systems Crew (bug fixes, UI)	2	100,000
Polish & Release	Content Lead (final pass)	1	50,000
Total		25	1,250,000

Buffer / overrun plan: If usage runs ahead of pace, scope sessions more narrowly to a single agent's file/system instead of whole-feature passes, cut the Encounter Agent's room count for the first playable pass (fewer rooms, same win condition), and drop the Testing phase's tuning passes to only the highest-impact balance flags from the QA/Balance Agent.

4.4 Constraints

Constraint: Unity 6.4 API drift from prototype-era code

What it is: My prototype build was originally written against an older Unity version; several APIs it relies on (input handling, rendering, some UI calls) changed by Unity 6.4.

Why it binds: I hit this directly upgrading my own prototype — copying old patterns straight into a Unity 6.4 project produced compile errors and deprecated-API warnings that had to be resolved by hand before any new system could be safely built on top.

What we do about it: The core grid/turn framework is already reconciled to Unity 6.4 going into this project. From here, the Systems Lead is responsible for flagging any prototype-era pattern that touches a changed API before an agent implements it, instead of porting old snippets wholesale.

Constraint: Agent context window vs. whole-project state

What it is: A coding agent working in the Unity project can't hold the entire project's state in context on every call — large scenes, all scripts, and all assets don't fit.

Why it binds: Tasks handed to a crew as “update the dungeon” without a narrower scope risk incomplete or conflicting edits across files the agent didn't actually see.

What we do about it: Work is scoped agent-by-agent to single systems or data files per session (the grid loader, the torch timer, one room's definition), using the clean interfaces already established in the upgraded framework so an agent only needs the relevant script and its direct dependencies in context, not the whole project.

4.5 Risks

- HeroQuest-style room-by-room flow plus grid combat is more moving parts than a pure grid-tactics game; mitigation: cut to the single dungeon wing scope in 4.1 and defer everything in the stretch goals list.

- Free asset packs sourced by the Art/Asset Agent may not share a consistent style across a full room set; mitigation: lock a small shared palette and tile-size spec early, and have the Content Lead reject a pack back to the agent if it clashes before it's wired into more than one room.
 - QA/Balance Agent suggestions could overfit to a single playtest run; mitigation: I only apply a tuning change after it shows up across at least two separate playtest sessions.
-

Appendix

Resolved: for the MVP, the party starts each delve with 2 torches plus the Priest's Light spell as a third light source. No other light-refill items are available before the delve begins — any further torches or light sources must be found and picked up during the crawl itself. This sidesteps needing a pre-delve shop or the town hub stretch goal for the MVP build entirely; that stretch goal, if built later, would be the point where purchasable Light-refill items get reconsidered.